

Figura 17.6. Lista de enteros: a) array; b) lista enlazada.

Los elementos de una lista están conectados o “enlazados” mediante sus campos enlace o puntero. La Figura 17.6 muestra un array (vector) y una lista enlazada, que representan ambas una lista de enteros.

Los componentes de un nodo se llaman *campos*. Un nodo tiene al menos un campo *datos* o *valor* y un campo *enlace* o *siguiente*. El campo enlace apunta (indica la dirección) al siguiente nodo de la lista. Existe una marca para fin de lista, que es la constante —ya conocida— *nil*, también representada por una barra inclinada o el signo eléctrico de tierra (Fig. 17.7).

El campo datos puede contener cualquier tipo estándar o definido por el usuario. El campo enlace contiene el punto al siguiente elemento de la lista.

Formato de declaración de un nodo (por ejemplo, de enteros)

```

type
  PunteroNodo = ^TipoNodo;
  TipoNodo   = record
    Info : integer;
    Enlace : PunteroNodo;
  end;

```

Observe que la definición del tipo puntero *PunteroNodo*

PunteroNodo = ^*TipoNodo*

precede a la definición del registro tipo *TipoNodo*. Esta es la única situación en que está permitido utilizar un identificador (*TipoNodo*) antes de ser definido.

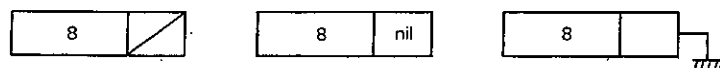


Figura 17.7. Símbolos del elemento *nil*.

17.5.1. Implementación de listas enlazadas con arrays

Existen dos métodos comúnmente utilizados para *implementar* (realizar) listas enlazadas. En los lenguajes de programación clásicos, las listas enlazadas se implementan utilizando arrays; los nodos se almacenan en arrays paralelos o arrays de registros, mientras que en los lenguajes evolucionados como Pascal se realizan con punteros.

El valor del nodo se almacena en un campo y el subíndice del siguiente elemento de la lista se almacena en el campo puntero. Un valor puntero de 0 indica el fin de la lista.

Una de las desventajas de almacenar listas en arrays es que se debe declarar un array lo bastante grande para contener la lista más grande posible con independencia del número de nodos que se utilicen realmente. En Pascal se pueden utilizar variables puntero y asignación dinámica para asegurar el espacio suficiente para la lista y su diseño es mucho más flexible y potente que mediante arrays.

17.5.2. Manipulación de listas enlazadas con punteros

Las operaciones básicas necesarias para manipular listas enlazadas son:

- *inicialización*
- *creación*
- *recorrido*
- *inserción*
- *borrado*
- *búsqueda*

Analicemos algunas de las opciones más significativas.

Inicialización

La primera operación a realizar con una lista enlazada es la *inicialización*. Esta operación constituye una lista vacía.

```

procedure Inicializar (var Lista: TipoLista {salida, lista lineal
                      enlazada});
begin
  {TipoLista, puntero a un nodo}
  L := nil {fija el puntero del primer nodo a nil}
end;

```

Creación

La construcción de una lista enlazada comienza con una lista vacía, insertando un nuevo elemento (nodo) al principio de una lista. El procedimiento funciona insertando al principio de una lista vacía [Fig. 17.8 a)], o bien insertando al principio de una lista no vacía.

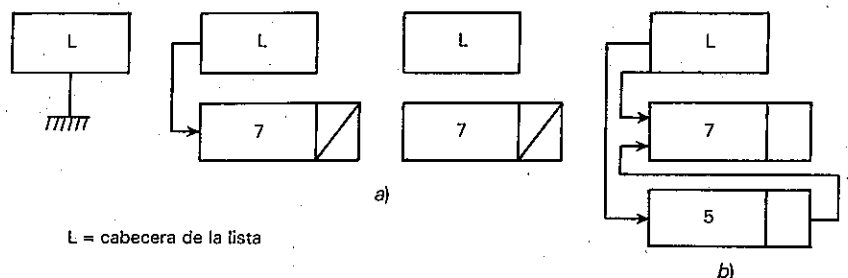


Figura 17.8. a) Inserción al principio de una lista vacía (L = nil). b) Inserción al principio de una lista no vacía.

En primer lugar se debe declarar un nodo tipo registro, con dos campos, uno de tipo cadena (info) y otro de tipo enlace que es un puntero.

```

type
  TipoInfo = string [9];
  TipoLista = ^TipoNodo;
  TipoNodo = record
    Info : TipoInfo;
    Enlace : TipoLista;
  end;

var
  Lista : TipoLista;

```

A continuación se debe declarar un procedimiento *Insertar*, al que se llamará cada vez que se desee insertar un nuevo nodo (elemento) en la lista. El procedimiento *Insertar* tiene dos variantes: 1) insertar al principio de una lista enlazada *InsertarAntes*; 2) insertar al final de una lista enlazada *InsertarDespués*.

Insertar al principio de la lista

```

procedure InsertarAntes (var Lista: TipoLista; {entrada/salida}
  Valor : TipoInfo { entrada de
                    informacion });
{ Inserta valor en un nuevo nodo al principio de la lista enlazada
  apuntada por el puntero Lista }
var
  Aux : TipoLista;
begin
  New (Aux); {obtiene un nuevo nodo}
  Aux^.info := Valor; { almacena la informacion en el nuevo
                      nodo }
  Aux^.enlace := Lista; { inserta el nuevo nodo a principio de
                        lista }
  Lista := Aux { Lista apunta al primer nodo de la
               lista }
end;

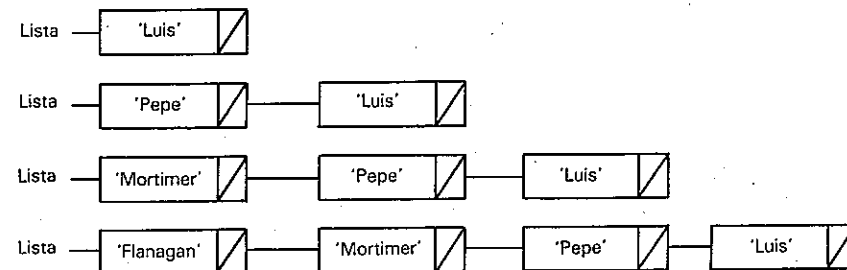
```

Ejemplo 17.6

Construcción de una lista enlazada de cuatro nodos.

La secuencia de procedimientos necesaria es:

1. Inicializar (Lista);
2. InsertarAntes (Lista, 'Luis');
3. InsertarAntes (Lista, 'Pepe');
4. InsertarAntes (Lista, 'Mortimer');
5. InsertarAntes (Lista, 'Flanagan');



Insertar al final de la lista

```

procedure InsertarDespués (var Lista: TipoLista; Valor: TipoInfo);
{insertar valor en un nuevo nodo, después del nodo apuntado por
 lista}
var
  Aux : TipoLista;
begin
  New (Aux); {crear nuevo nodo}
  Aux^.info := Valor; {inicializa contenido nuevo nodo}
  Aux^.enlace := nil;
  {si la lista original esta vacia, entonces el primer puntero
  apunta al nuevo nodo}
  if Lista = nil
  then
    Lista := Aux
  else
    begin
      Aux^.enlace := Lista.enlace;
      Lista^.enlace := Aux
    end
end;

```

Recorrido de una lista

La operación de imprimir los datos de una lista se conoce como *recorrido*. Se debe comenzar por la cabecera de la lista y seguir con todos los puntos de la lista, imprimiendo todos los campos info (datos).

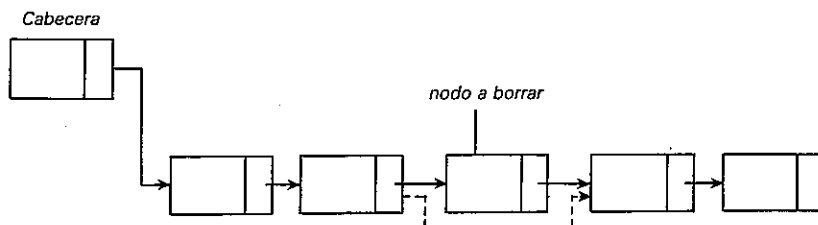
```

procedure ImprimirLista (Lista : TipoLista);
{imprimir los valores de cada nodo apuntado por lista}
begin
  while lista <> nil do
    begin
      WriteLn (Lista^.info);      { imprime el valor del nodo }
      Lista := Lista^.enlace      { avanza al siguiente nodo }
    end
  end; {ImprimirLista}

```

Eliminación de un nodo

Borrar o eliminar el nodo al que se está apuntando exige conectar el nodo precedente al nodo después del que se desea borrar.



```

procedure Borrar (var Lista: TipoLista; P1: TipoLista);
{P1, elemento a borrar}
var
  Q1 : TipoLista; {Puntero que se mueve por la lista}
begin
  if P1 = Lista      {si es el primer elemento}
  then Lista := Lista^.Enlace {Eliminar P1}
  else {sino encontrar elemento antes de P1}
  begin
    Q1 := Lista;
    while Q1^.Enlace > P1 do
      Q1 := Q1^.Enlace;
    Q1^.enlace := P1^.Enlace {eliminar P1}
  end; {else}
  Dispose (P1) {liberar memoria}
end; {Borrar}

```

Problema 17.1

Crear una lista enlazada, insertar nuevos nodos (al final de la lista) y recorrer la lista.

Análisis

Se necesitan tres punteros:

primero apunta hacia el primer elemento de la lista
 último apunta hacia el último elemento de la lista
 nuevo elemento a insertar, apuntado por nuevo

```

program Dinamico;
type
  Cad9      = string [9];
  Listaptr  = ^Nodo;
  Nodo      = record
    Nombre : Cad9;
    Enlace : Listaptr;
  end;

var
  Primero, Ultimo : Listaptr;
  Cadena          : Cad9;

procedure Insertar (var Final : Listaptr; Elemento : cad9);
var
  Nuevo : Listaptr;
begin
  New (Nuevo);
  Nuevo^.Nombre := Elemento;
  Nuevo^.Enlace := nil;
  Final^.Enlace := Nuevo;
  Final := Nuevo
end;

procedure Leer (Inicio : Listaptr);
begin
  while Inicio <> nil do
    begin
      Write (Inicio^.Nombre, ' ');
      Inicio := Inicio^.Enlace
    end
  end;
end;

begin {programa principal}
  New (Primero);
  Primero^.Nombre := 'comienzo';
  Primero^.Enlace := nil;
  Ultimo := Primero;
  WriteLn ('Insercion : introduzca palabras (Fin/Terminar)');
  repeat
    ReadLn (Cadena);
    Insertar (Ultimo, Cadena)
  until cadena = 'Fin';
  Leer (Primero)
end.

```

17.6. LISTAS CIRCULARES

Las listas son estructuras muy ricas y variadas. Hemos visto las más simples: las listas lineales. Veamos ahora otros tipos de listas, similares a las ya estudiadas: las *listas circulares*. Una *lista lineal enlazada circularmente*, o simplemente una *lista circular*, es una

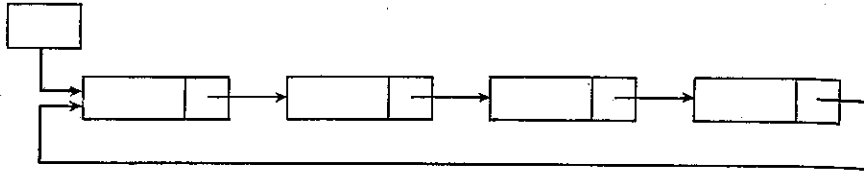


Figura 17.9. Lista circular.

lista en la que el puntero siguiente al "último" elemento apunta hacia el primer elemento o nodo (Fig. 17.9). En estas listas no existe ni primero ni último elementos, aunque se debe elegir obligatoriamente un puntero para referenciar la lista.

Esta lista presenta la gran ventaja de que cada nodo en una lista circular es accesible desde cualquier nodo. Tiene el inconveniente de que requiere un diseño cuidadoso para evitar caer en un bucle infinito.

Procedimiento Insertar

El procedimiento *insertar* describe la inserción del elemento apuntado por *nuevo* a continuación del elemento apuntado después.

```
procedure Insertar (var Despues : TipoLista; Nuevo : TipoLista);
begin
  if Despues = nil
  then
    Despues := Nuevo
  else
    Nuevo^.Enlace := Despues^.Enlace;
    Despues^.Enlace := Nuevo
  end;
end;
```

Procedimiento Eliminar

El procedimiento *eliminar* borra el elemento que sigue al elemento designado por el puntero *despues*.

```
procedure Suprimir (var Despues: TipoLista);
{ elimina el elemento que sigue al apuntado por Despues }
var
  Aux : TipoLista;
begin
  if Despues <> nil
  then
    begin
      Aux := Despues^.Enlace;
      if Despues = Aux
      then Despues := nil
      else Despues^.Enlace := Aux^.Enlace;
      Dispose (Aux)
    end
  end
end;
```

17.6.1. Listas doblemente enlazadas

Hasta este momento, las listas enlazadas se han recorrido en un solo sentido, de izquierda a derecha. En muchas aplicaciones se requiere recorrer la lista en ambas direcciones. Estos recorridos se pueden conseguir manteniendo dos campos de enlace en cada nodo, en lugar de uno. Estos enlaces (punteros) se utilizan para denotar la dirección del predecesor y sucesor de un nodo dado. El predecesor se llama *enlace izquierdo* y el sucesor *enlace derecho*. Una lista cuya estructura de nodos contiene dos campos de enlace se denominará *lista lineal doblemente enlazada* (Fig. 17.10).

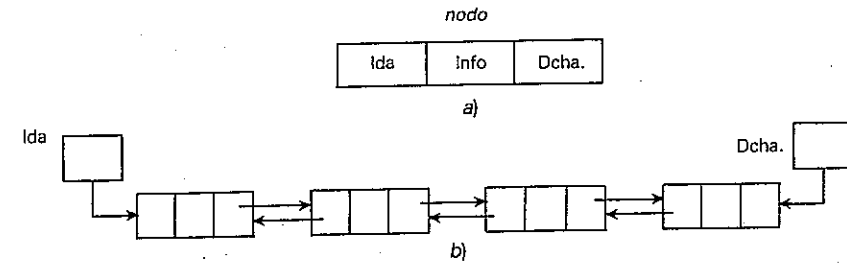


Figura 17.10. Lista lineal doblemente enlazada.

Las siguientes declaraciones describen un nodo general de una lista doblemente enlazada:

```
type
  TipoListaPtr = ^TipoNodo;
  TipoLista = record
    Ida, Dcha: TipoListaPtr
  end;
  TipoNodo = record
    PtrI : TipoListaPtr;
    Info : TipoInfo;
    PtrD : TipoListaPtr
  end;
```

Por último, se puede considerar un nuevo tipo de lista: la lista circular doblemente enlazada.

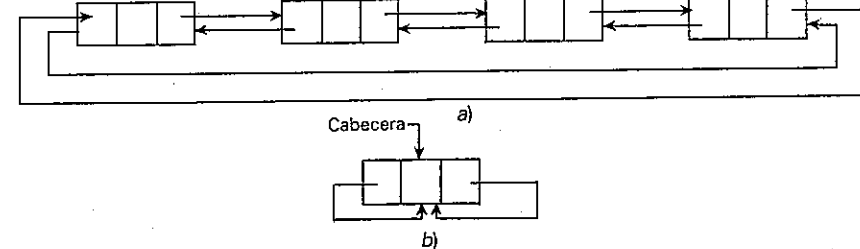
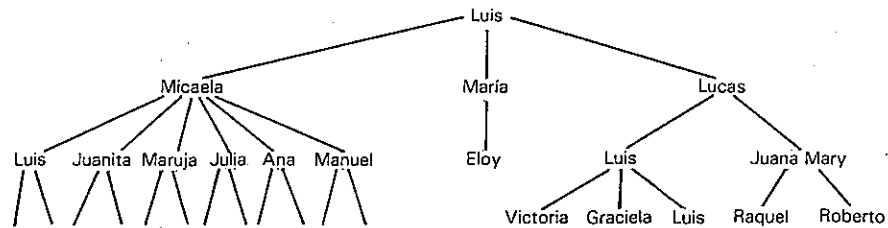


Figura 17.11. Lista circular doblemente enlazada con un nodo cabecera.

17.7. ARBOLES

Las listas enlazadas son estructuras de datos lineales que si bien ofrecen una gran flexibilidad de diseño, determinados problemas son difíciles o, en su caso, lentos de resolver; por ejemplo, el caso de la búsqueda de un determinado elemento que exige, como el lector sabe, un recorrido secuencial que ralentiza el proceso. Los *árboles* son estructuras que permiten representar datos que tienen enlaces jerárquicos entre ellos. Existen diferentes tipos de árboles, pero en nuestro estudio nos limitaremos a los árboles binarios.

La representación más corriente de un árbol es la que recuerda a los árboles genealógicos



Un *árbol* se define como un conjunto de uno o más nodos, tales que:

1. Existe un nodo especial llamado *raíz*.
2. Los restantes nodos se dividen en $m \geq 0$ conjuntos disjuntos $T_1, T_2, \dots, T_n$, cada uno de los cuales es un árbol. Cada T_i ($1 \leq i \leq n$) se denomina *subárbol* del raíz.

En la Figura 17.12 se representa un árbol de doce nodos.

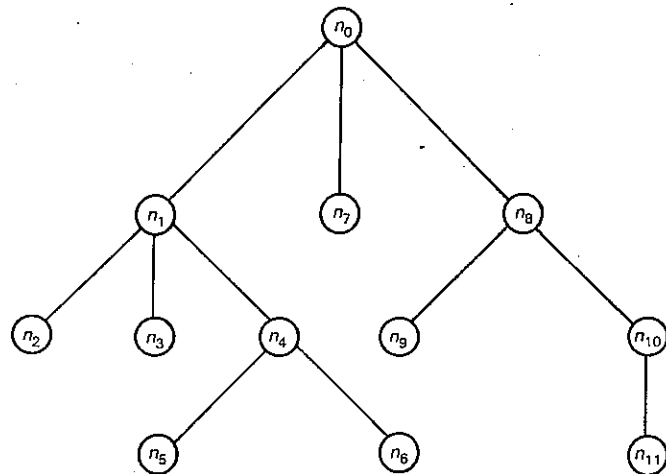


Figura 17.12. Árbol de doce nodos ($n_0 \dots n_{12}$).

En la terminología de árboles se destacan los siguientes conceptos:

- Nodo raíz o raíz del árbol* : elemento superior (cima) del árbol (n_0)
- Nodo terminal (hoja o rama)* : un nodo que no tiene subárboles
- Nodo interior (bifurcación)* : nodos con uno o dos subárboles
- Bosque* : los nodos de cada nivel están ordenados (la permutación de sus nodos origina un árbol diferente)

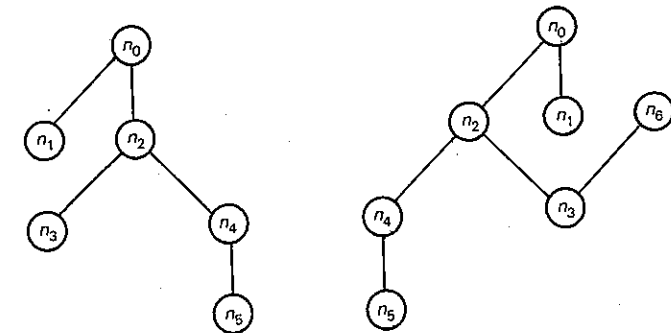


Figura 17.13. Organización de nodos.

- Descendiente* : un nodo situado directamente debajo de otro
- Ascendente* : un nodo situado directamente encima de otro
- Hoja* : elemento o nodo terminal, sin descendiente

Los nodos se organizan en niveles. La raíz de un árbol se define como *nivel*; cada descendiente directo pertenece a un nivel superior. El número más elevado de descendientes directos de un nodo define el *grado* de un árbol (número de subárboles, en la Figura 17.12 el árbol tiene un nodo de nivel 1, tres nodos de nivel 2, cinco nodos de nivel 3 y tres nodos de nivel 4). El nodo raíz tiene un grado de 3 y n_{10} es de grado 1. El grado de una hoja es 0. El nivel más elevado que se encuentra en un árbol determina su *profundidad* (árbol de la Figura 17.13). Como ya se ha considerado antes, centraremos nuestro estudio en los tipos de árboles denominados árboles binarios.

17.8. ARBOL BINARIO

Un árbol binario es un árbol ordenado de grado 2.

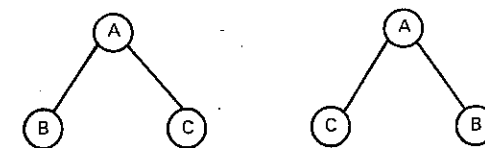


Figura 17.14. Árboles binarios.

En general, la definición más completa es:

Un árbol binario es un conjunto finito de m ($m \geq 0$) nodos, que es o bien vacío ($m = 0$) o consta de un nodo raíz con dos subárboles binarios llamados subárbol izquierdo y subárbol derecho. Un árbol binario de cero nodos se dice que está vacío.

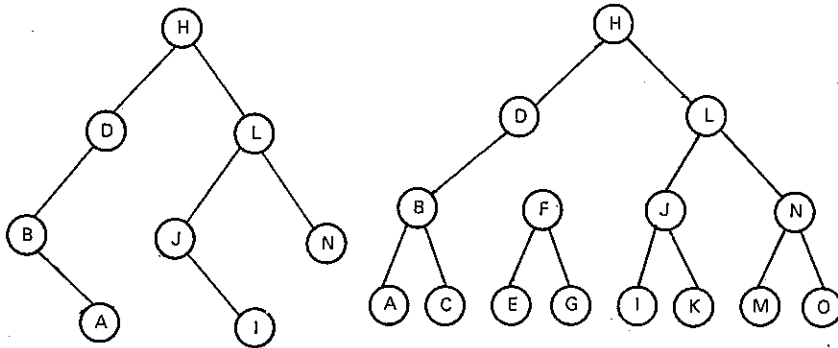


Figura 17.15. Árboles binarios.

La representación de un árbol binario requiere nodos que deben contener al menos tres campos:

- Un campo de datos: *Info* (cualquier tipo de dato predefinido o definido por el usuario, incluyendo punteros a otras estructuras).
- Un puntero al subárbol izquierdo del nodo (que puede ser *nil*).
- Un puntero al subárbol derecho del nodo (que puede ser *nil*).

Se necesitan dos punteros, uno hacia el subárbol izquierdo y otro hacia el subárbol derecho. La representación más extendida del árbol binario es mediante una lista o almacenamiento enlazado.

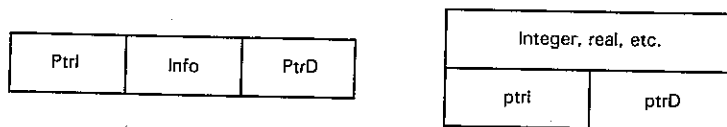


Figura 17.16. Diagrama de un nodo de árbol binario.

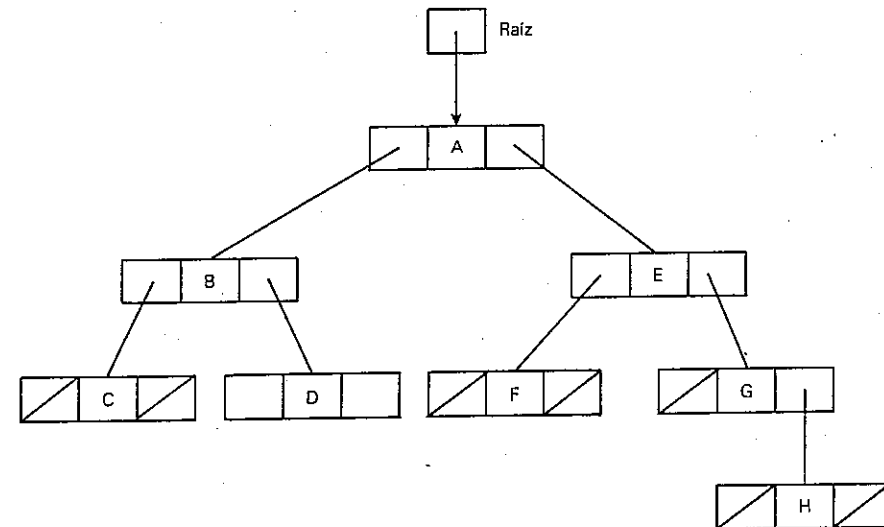
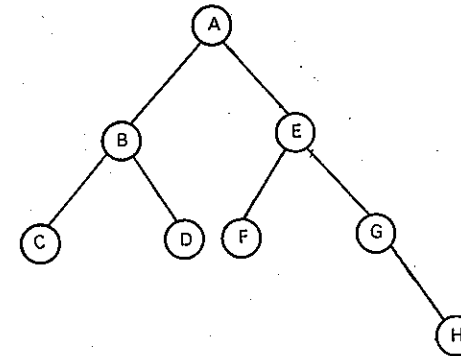


Figura 17.17. Árbol binario y su representación enlazada.

La declaración de las definiciones del tipo árbol serán:

```
type
{ declaraciones previas }
TipoInfo = ...
PtrArbol = ^NodoArbol;
NodoArbol = record
    Info : TipoInfo; { tipo previamente declarado }
    PtrI, PtrD : PtrArbol
end;
```

Ejemplos

1. Arbol con informaciones en los nodos de tipo carácter; se suele utilizar un puntero, raíz, hacia la raíz propiamente dicha del árbol.

```

type
  Info = char;
  Ptr = ^Nodo;
  Nodo = record
    El_Datos : Info;
    Ida      : Ptr;
    Dcha     : Ptr;
  end;
var
  Raiz : Ptr;
  Datos : Info;

```

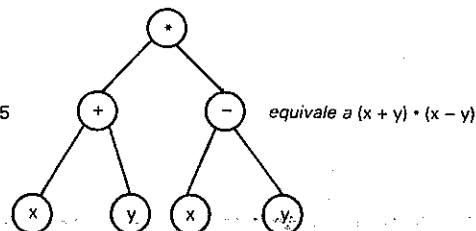
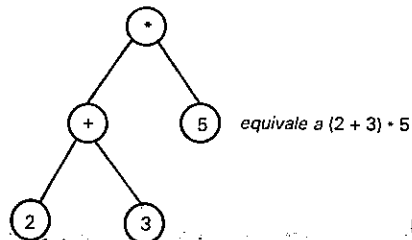
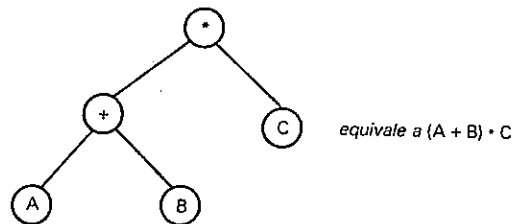
2. Arbol con información en los nodos de tipo cadena.

```

type
  Cad3 = string[3];
  Ptr = ^Nodo;
  Nodo = record
    Palabra : Cad3;
    Ida, Dcha: Ptr;
  end;

```

Los árboles pueden ser utilizados para representación de expresiones en memoria. Por ejemplo:



17.8.1. Inserción en un árbol

La inserción de un nuevo nodo en un árbol se resuelve con el procedimiento Insertar (caso del ejemplo 1).

```

procedure Insertar (X: info; var P: ptr);
begin
  if P = nil then
    begin
      New (P);
      with P^ do
        begin
          ElDatos := X;
          Ida      := nil;
          Dcha     := nil;
        end
      end
    end
  else
    if X < P^.ElDatos
    then Insertar (X, P^.Ida)
    else Insertar (X, P^.Dcha)
  end;
end;

```

17.8.2. Recorrido de un árbol

Una vez que el árbol está construido será necesario consultar las informaciones que contiene. Esta consulta (*recorrido del árbol*) exige seguir algún determinado orden. En el caso de las listas es claro: principio a fin. En el caso del árbol, el recorrido se puede efectuar de diversas maneras:

- en orden (*inorder*),
- preorden (*preorder*),
- postorden (*postorder*).

Los pasos de cada algoritmo son:

Recorrido en orden (IRD)

1. Si el árbol está vacío, salir (fin).
2. Recorrer el subárbol izquierdo (I).
3. Visitar (procesar) el nodo raíz (R).
4. Recorrer el subárbol derecho (D).

Recorrido preorden (RID)

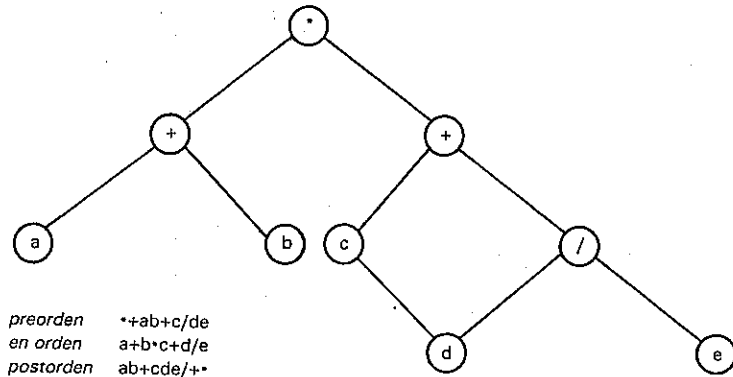
1. Si el árbol está vacío, salir (fin).
2. Visitar (procesar) el modo raíz (R).
3. Recorrer el subárbol izquierdo (I).
4. Recorrer el subárbol derecho (D).

Recorrido postorden (IDR)

1. Si el árbol está vacío, salir (fin).
2. Recorrer el subárbol izquierdo (I).
3. Recorrer el subárbol derecho (D).
4. Visitar (procesar) el nodo raíz (R).

Ejemplo

Obtener la expresión equivalente del árbol.



17.8.3. Visualizar contenido de un árbol

La codificación de los algoritmos correspondientes, según los tres métodos citados, y para el caso del árbol del ejemplo, es:

```
procedure Preorden (P : Ptr);
begin
  if P <> nil then
    begin
      Write (P^.ElDato : 6);
      Preorden (P^.Ida);
      Preorden (P^.Dcha);
    end
  end;
```

```
procedure Inorden (P : Ptr);
begin
  if P <> nil then
    begin
      Inorden (P^.Ida);
      Write (P^.ElDato : 6);
    end
  end;
```

```
procedure Postorden (P : Ptr);
begin
  if P <> nil then
    begin
      Postorden (P^.Ida);
      Postorden (P^.Dcha);
      Write (P^.ElDato : 6);
    end
  end;
```

17.8.4. Borrado de nodos de un árbol

La operación de borrado, al igual que en otras estructuras, consiste en eliminar el nodo de clave *C* de un árbol con los nodos ordenados. Se pueden presentar tres casos:

1. El nodo de clave *C* no se encuentra.
2. El nodo de clave *C* tiene 0 ó 1 subárbol.
3. El nodo de clave *C* tiene dos subárboles.

En el primer caso no se necesita realizar ninguna acción; en el segundo caso bastaría con sustituir (si existiera) el nodo a eliminar por su hijo; y por último, en el tercer caso, que es el más complejo, se debe sustituir el nodo a borrar por su descendiente más a la izquierda de su subárbol derecho o por el más a la derecha de su subárbol izquierdo. Un ejemplo gráfico se indica en la Figura 17.18, donde se sustituye el nodo a borrar por el descendiente más a la derecha de su subárbol izquierdo.

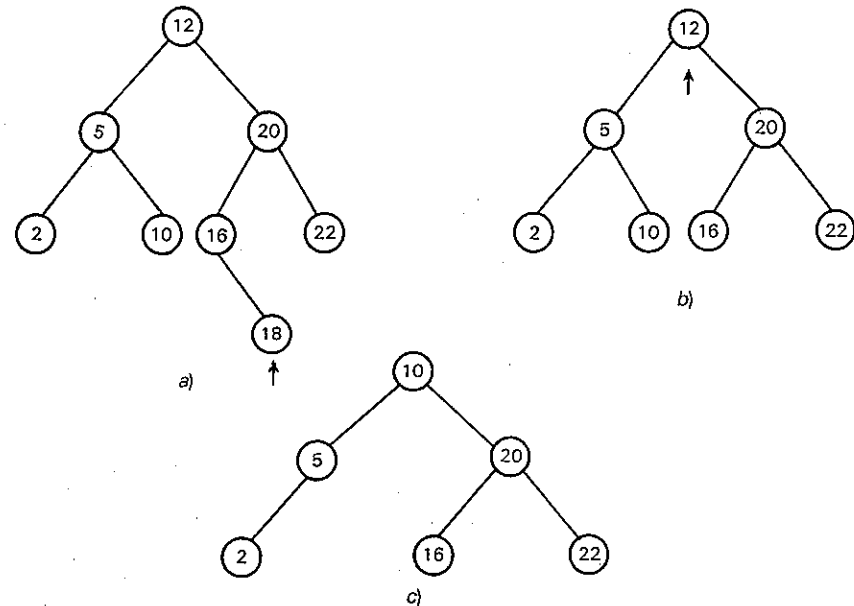


Figura 17.18. Borrado de un nodo con dos subárboles.

```

procedure Eliminar (C : info; var P: Ptr);
var
  Aux1 : Ptr;

procedure Borra (var P1 : Ptr);
begin
  if P1^.Dcha <> nil then
    Borra (P1.Dcha)
  else
    begin
      Aux1^.ElDato := P1^.ElDato;
      Aux1         := P1;
      P1           := P1^.Izdo
    end
  end;
begin
  if P = nil then
    WriteLn ('no se encuentra')
  else
    if C < P^.ElDato then
      Eliminar (C, P^.Izda)
    else
      if C > P^.ElDato then
        Eliminar (C, P^.Dcha)
      else
        begin
          Aux1 := P;
          if Aux1^.Dcha = nil then
            P := Aux1^.Izda
          else
            if Aux1^.Izda = nil then
              P := Aux1^.Dcha
            else
              Borra (Aux1^.Izdo)
            end
          end
        end
      end;
  {Eliminar}
end;

```

Como ha podido observar el lector, los procedimientos son recursivos por la naturaleza recursiva de la ley de formación del árbol.

Se deja como ejercicio al lector el diseño de los procedimientos *suprimir* (un nodo del árbol) y *buscar* (un nodo del árbol).

17.9. PUESTA A PUNTO DE PROGRAMAS

Técnicas de programación

- Un puntero es la dirección de una posición de memoria en la que sólo un valor de ese tipo se puede almacenar.
- Para verificar si una expresión con punteros es correcta, verificar el tipo para ver si es un tipo puntero o un tipo nodo.

- Las ventajas de una lista enlazada sobre un array es que la lista enlazada puede crecer y contraerse, y que es fácil insertar o borrar un valor (nodo) en el centro de una lista enlazada.
- La constante *nil* se utiliza para marcar puntos finales, tal como el fin de una lista enlazada, y denotar estructuras vacías, tales como una lista enlazada vacía o un árbol vacío.
- Verificar siempre que los procedimientos para manipular estructuras de datos dinámicas, tales como listas enlazadas y árboles, funcionan correctamente para la estructura vacía.
- Los únicos valores que se pueden asignar a una variable puntero utilizando una sentencia de asignación son el valor *nil* o el valor de otra variable puntero del mismo tipo.
- Las operaciones aritméticas no se pueden ejecutar sobre punteros.
- Los valores punteros no pueden ser comparados con los operadores relacionales *<*, *<=*, *>* y *>=*. Sólo están permitidos los operadores *=* y *<>*.
- Los valores punteros no pueden leerse o visualizarse.
- Los punteros pueden ser utilizados como parámetros en funciones y procedimientos, pero los parámetros correspondientes actuales y formales no pueden estar limitados a tipos diferentes.
- El valor de un puntero P es la dirección de una posición de memoria, mientras que el valor de P^ es el dato almacenado en esa posición.
- Un puntero indefinido no es lo mismo que un puntero *nil*.
- Si P es un puntero que es indefinido o *nil*, cualquier intento de utilizar P^ es un error.

Errores típicos de programación

- La depuración de un programa que implique variables de puntero puede ser, a veces, difícil, debido al hecho que el valor de una variable puntero representa una dirección de la celda de memoria y normalmente no se puede imprimir. En consecuencia, si un valor puntero no es válido o incorrecto, puede ser difícil encontrar cuál es el error.
- Asegurarse que el símbolo ^ sigue a cada variable de puntero utilizada para designar un nodo particular o registro. El ^ debe escribirse siempre cuando se desea manipular un campo de un nodo.
- Otra posible causa de error es el intento de referenciar un campo de un nodo apuntado por un puntero cuyo valor es *nil*. Esta operación no es legal, ya que el puntero *nil* es un valor especial utilizado para indicar el fin de una lista.

RESUMEN

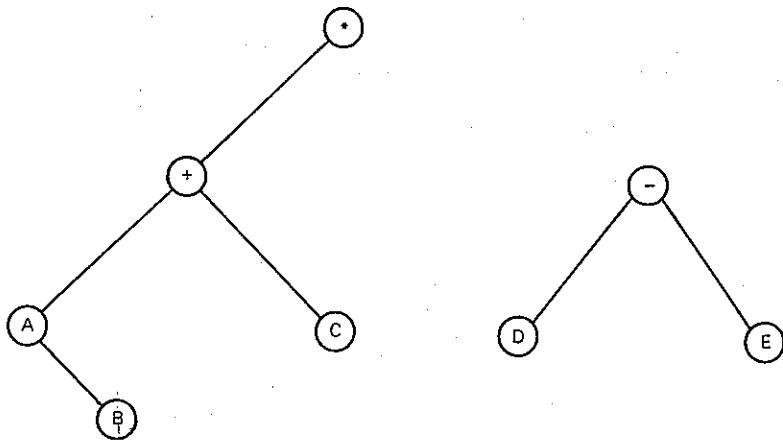
En este capítulo se introduce un nuevo tipo de estructuras de datos: *las listas lineales enlazadas*. Al contrario que los arrays, el tamaño de una lista enlazada puede variar durante la ejecución de un algoritmo (programa). En las estructuras de datos elementales (arrays) los elementos se almacenaban en memoria de modo adyacente, mientras que en el caso de las listas enlazadas los elementos *lógicamente contiguos* no tienen por qué ser adyacentes *físicamente*. Se describen en el capítulo diferentes estructuras enlazadas: listas enlazadas simples, listas enlazadas circularmente y listas doblemente enlazadas.

Se examinan también en este capítulo las estructuras más importantes de datos no lineales: los árboles. Dado que los árboles binarios son más fáciles de representar y manipular que los árboles en general, se presta especial atención al concepto tratamiento de los árboles binarios.

EJERCICIOS

1. Dada una lista lineal enlazada sencilla, cuyos nodos tienen la estructura (Info y Enlace), diseñar un procedimiento que cuente el número de nodos de la lista.
2. Escribir un procedimiento que modifique el campo Info del nodo n -ésimo de una lista enlazada por un valor dado t .
3. Construir un procedimiento que realice una inserción inmediatamente antes del nodo n -ésimo de la lista.
4. Escribir un procedimiento que inserte un nodo al final de una lista circular.
5. Diseñar un procedimiento que inserte un nodo al final de una lista circular.
6. Escribir los procedimientos insertar y borrar un elemento en una lista circular.
7. ¿Cuántos árboles diferentes existen con tres nodos?
8. Dada la expresión de la siguiente línea, dibujar el árbol equivalente

$$[(x-y)*z]/(m+n^p)$$
9. Recorrer en los tres nodos el árbol siguiente:

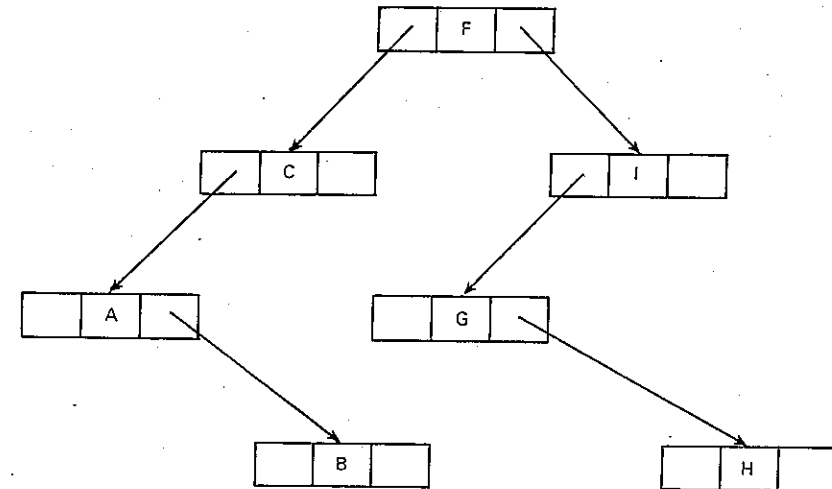


10. El recorrido en orden de un árbol binario produce la secuencia de nodos:

dfcgbca

Dibujar el árbol

11. Recorrer el árbol en los tres procedimientos:



12. Escribir procedimientos que permitan la eliminación y búsqueda de un nodo en una lista doblemente enlazada.

PROBLEMAS

1. Con la ayuda de las declaraciones siguientes

```

type
  Ptr = ^Nodo;
  Nodo = record
    Numero : integer;
    Enlace : Ptr;
  end;

var
  Primero, Ultimo, Nuevo : Ptr;

```

escribir un programa que cree una lista enlazada de números enteros. Cada número se sitúa en orden creciente. El programa visualizará el contenido de la lista.

2. Escribir un programa que ejecute las operaciones siguientes:
- a) Crear una lista enlazada con los tipos de datos siguientes:

```

type
  Ptr = ^Nodo;
  Nodo = record
    Info : string [30];
    Enlace : Ptr;
  end;

```

- b) Recorrido de la lista e impresión de sus elementos.